Location-Based Services, Summer Semester 2025

Msc. Geodetic Engineering

Task: Calculates Alternative routes for wheelchair navigation using Dijkstra's algorithm

Team Members: Jignesh Amadavadi, Abdul Ahad

Supervisors: Prof. Dr.-Ing. Jan-Henrik Haunert, MSc. Shiyazi Risvi

Institute of Geodesy and Geoinformation
University of Bonn

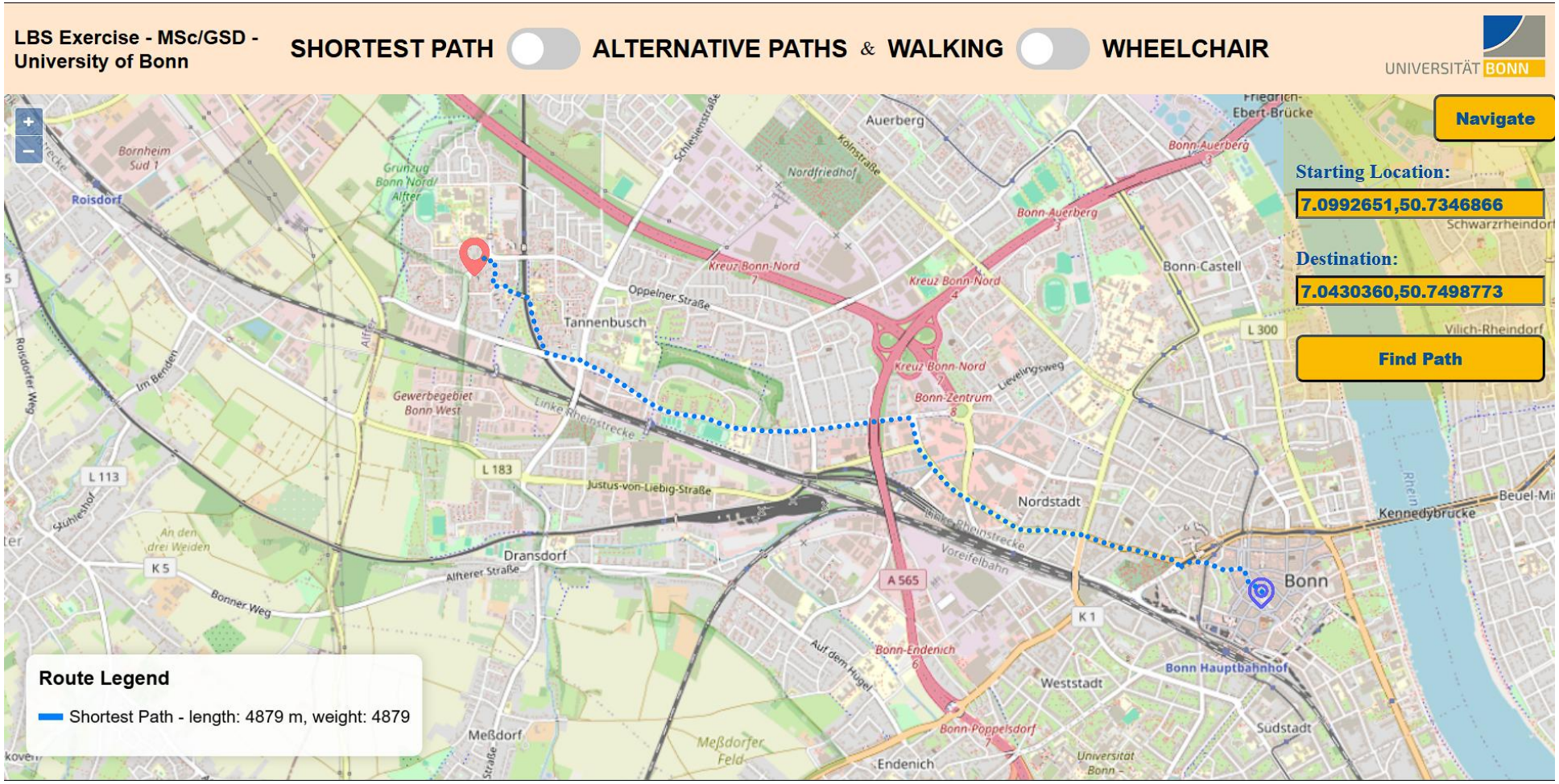
Task a: Visualize Shortest Walking Path

Familiarize Ourself with the Code

- **Data and Dependencies Download:**
  - Installed all related dependencies and loaded pedestrian road network provided by OSM using Geofabrik.
- **Study Area:**
  - Bonn City
- **Data Sources:**
  - OpenStreetMap and Digital Elevation Models (DEM)
- **Already Implemented:**
  - Shortest walking path already implemented.
  - Used Dijkstra’s algorithm with 'length' as weight.
  - Displayed optimal walking route in web interface.
  - Verified basic functionality of routing system.

Task a: Visualize Shortest Walking Path

Visualized Shortest Walking Path in Web Frontend



Task b: Data Preprocessing for Wheelchair Routing

Data Collection And Preprocess The Data For Shortest Path For Wheelchair Navigation

- Downloaded and used DEM to get elevation for each node
- Reprojected graph to match DEM CRS (Raster Projection) for accurate elevation computation
- Added elevation to each node and computed slope for edges
- Steps were kept in the graph and later penalized

Task c: Graph Preparation

Creating Routing Graph

- **Slope Calculation:**
  - $slope = \frac{Elevation\ end - Elevation\ start}{Horizontal\ Distance\ (length)}$
  - Uphill = Positive, Downhill = Negative
  - Slope calculated for forward direction (source to target) as well for the backward direction (-slope).
- Built a directed graph with elevation and slope attributes
- Included edge attributes: distance (length), slope (grade), and road type (highway type)
- Reprojected the graph to WGS 84 projection from raster projection (DEM CRS)
- Saved the graph to bonn.graphml for reuse
- Steps were kept in the graph and later penalized
- Base graph ready for both walking and wheelchair routing

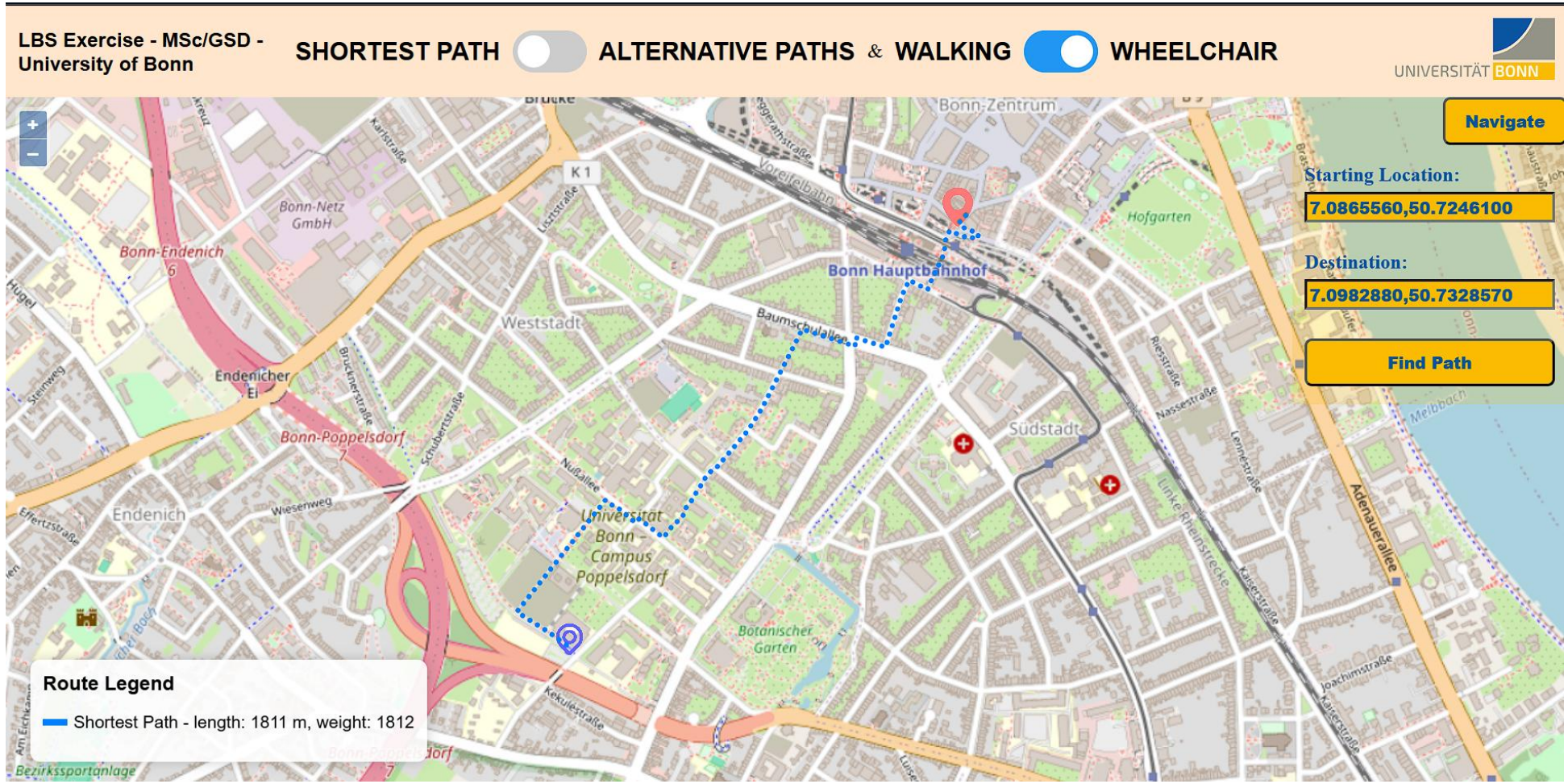
Task d: Visualize Shortest Wheelchair Path

Shortest path for Wheelchair Navigation

- **Applied custom Weights:**
  - Steps ⇨ Infinite weight (Not completely removed)
- **Designed Slope Penalty system:**
  - Uphill penalized more than downhill
- **Slope Penalty Calculation:**
  - If slope is positive, $slope\ penalty = slope^2$
  - If slope is negative, $slope\ penalty = 0.5\ * slope^2$
- **Weight Calculation:**
  - $weight = length\ * (1\ + slope\ penalty)$
- **Effect:**
  - Steep paths are not removed, but Dijkstra avoids them due to high cost

Task d: Visualize Shortest Wheelchair Path

Visualized Shortest Wheelchair Path in Web Frontend



Task e: Understand Constraints

Constraints Adopted From Abraham et al., 2013 Paper

- **Limited Sharing(γ):**

  Only a limited portion of the alternative path should overlap with the shortest path.

  Mathematically:

$$l(Opt \cap P) \leq \gamma \cdot l(Opt)$$

  - P is the alternative path.
  - Opt is the shortest path.
  - γ is typically between 0.3 and 0.7.
- **Why ?**
  - We don't want alternative paths that are nearly the same as the original.

Task e: Understand Constraints

Constraints Adopted From Abraham et al., 2013 Paper

- **Local Optimality(α) :**

  A path P is T-locally optimal if:

  Every subpath of length ≤ T is a shortest path.

  Mathematically:

$$T \leq \alpha \cdot l(Opt)$$

  - Opt is the shortest path.

- **Why ?**
  - To avoid silly or illogical detours in small parts of the alternative path.

Task e: Understand Constraints

Constraints Adopted From Abraham et al., 2013 Paper

- **Uniformly Bounded Stretch(ε) :**

  Every subpath of the alternative P must satisfy:

$$l(P') \leq (1 + \varepsilon) \cdot l(OptP')$$

  - Where P’ ⊆ P, and OptP’ is the optimal path between the same start/end nodes of P’.

- **Why ?**
  - To make sure that the alternative path isn’t too much longer than needed – not just the full alternative path but also the subpath of it.

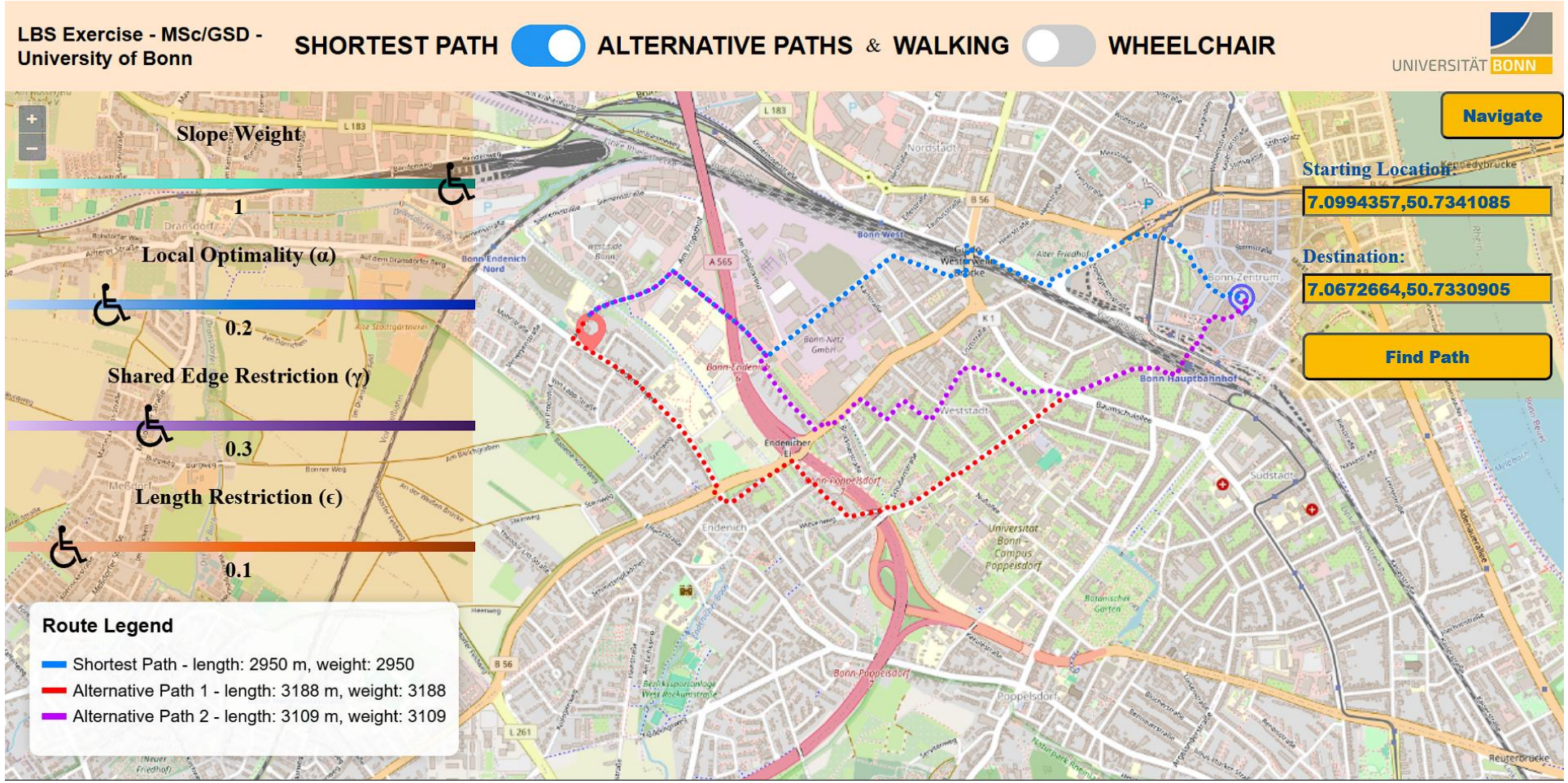
Task f: Implementation of Constraints

Implement Alternative Paths

- **Used single via-node approach (source → via → target)**
- **For each via-node candidate:**
  - Built full path: source → via → target
  - Applied 3 constraints from Abraham et al.:
    1. Limited Sharing (γ): Limit Overlap with shortest path
    2. Local Optimality (α): Subpaths near via – node must be shortest
    3. Bounded Stretch (ε): Full path must not be too long
  - Returned only paths that passes all 3 constraints.

Task g: Testing Alternative Paths

Alternative Paths for Walking mode

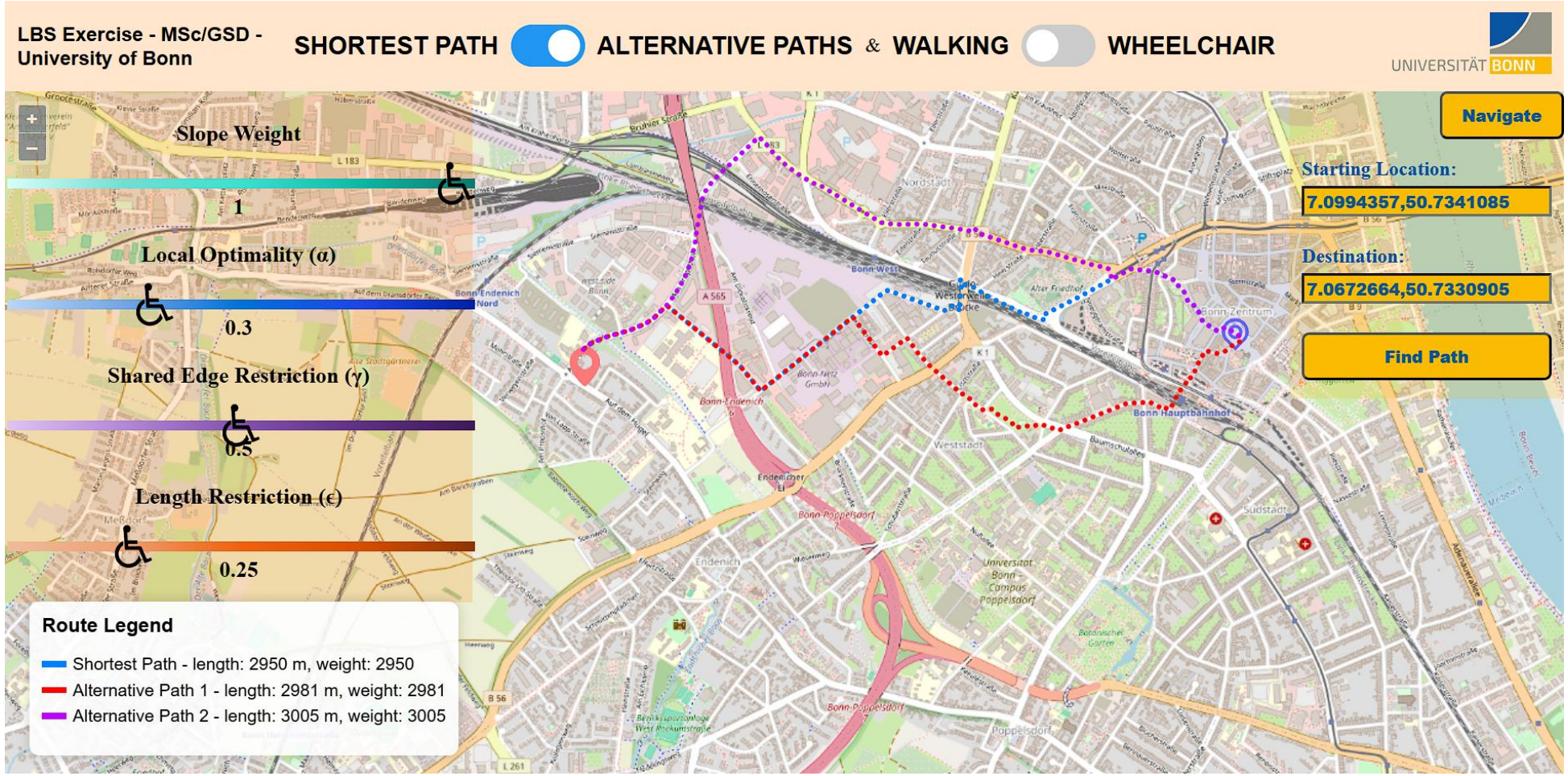


For,

$$\alpha = 0.2$$
$$\gamma = 0.3$$
$$\varepsilon = 0.1$$

Task g: Testing Alternative Paths

Alternative Paths for Walking mode

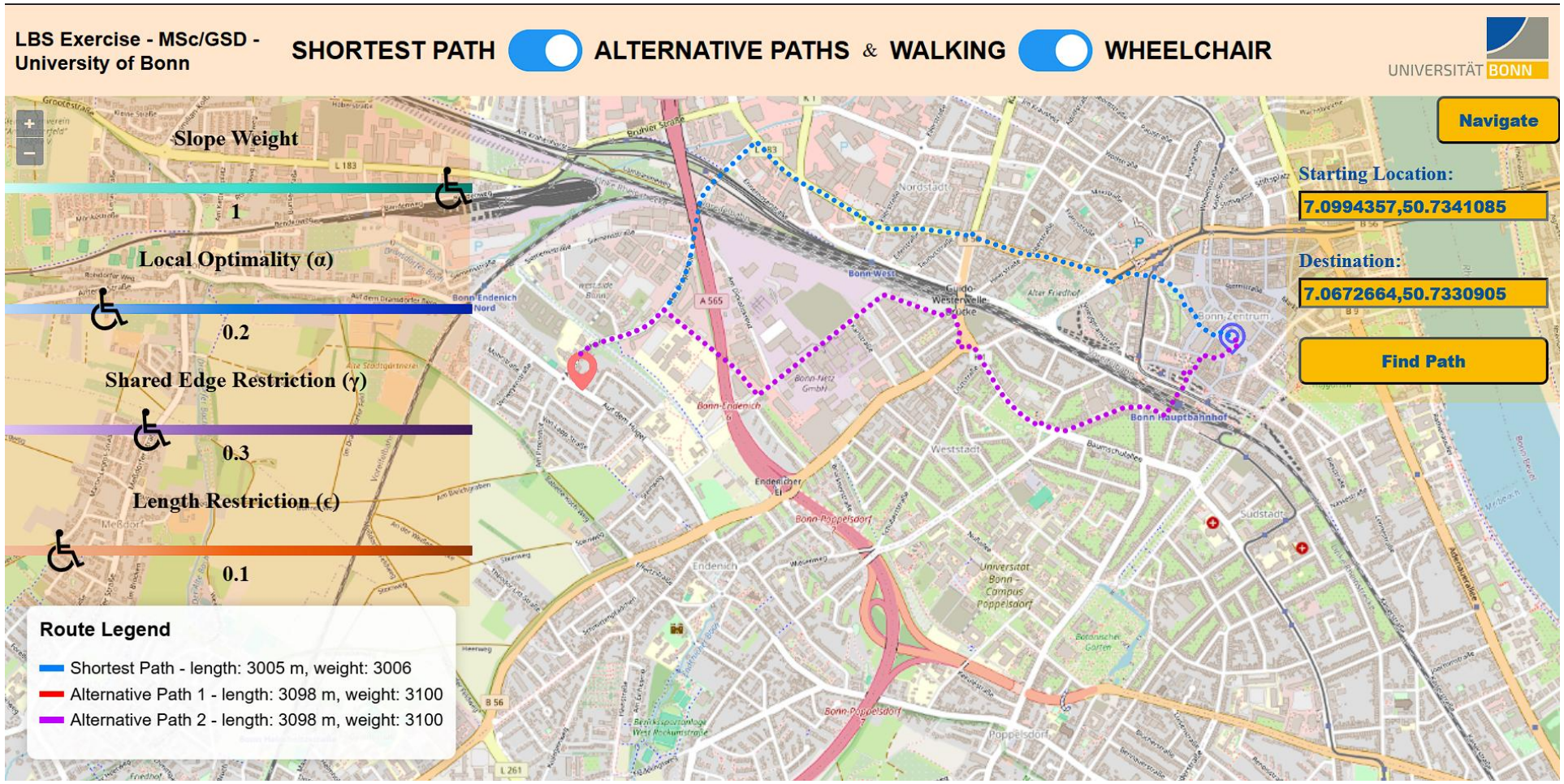


For,

$$\alpha = 0.3$$
$$\gamma = 0.5$$
$$\varepsilon = 0.25$$

Task g: Testing Alternative Paths

Alternative Paths for Wheelchair mode

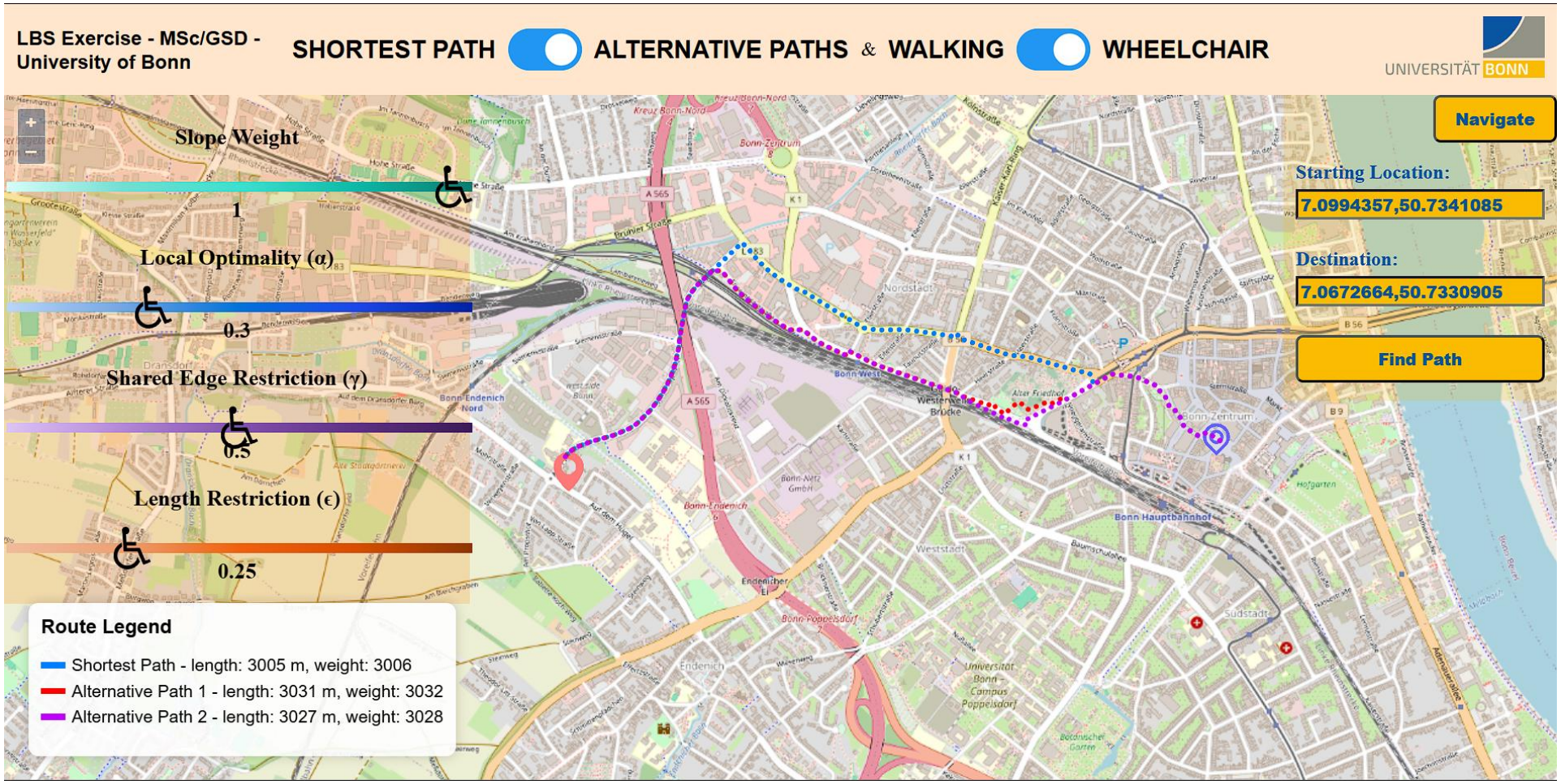


For,

$$\alpha = 0.2$$
$$\gamma = 0.3$$
$$\varepsilon = 0.1$$

Task g: Testing Alternative Paths

Alternative Paths for Wheelchair mode



For,

$$\alpha = 0.3$$
$$\gamma = 0.5$$
$$\varepsilon = 0.25$$

Task g: Testing Alternative Paths

Effect of parameter choices

Mode	α	γ	ε	No. of Alternative paths	Runtime (in seconds) for Shortest path	Runtime (in seconds) for Alternative paths
Wheelchair	0.2	0.3	0.1	1	0.886	5.389
	0.3	0.5	0.25	2	0.831	14.69
Walking	0.2	0.3	0.1	2	0.798	24.098
	0.3	0.5	0.25	2	0.802	24.075

Thank you